Clase 11: *Multithreading* en Python – Numba

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Jueves 16 de Abril del 2026

Clase de hoy

- ¿Cómo programar la ejecución de múltiples hilos en Python?

Tipos de memoria

La distinción principal en computación paralela: **memoria compartida** vs **memoria distribuida**.

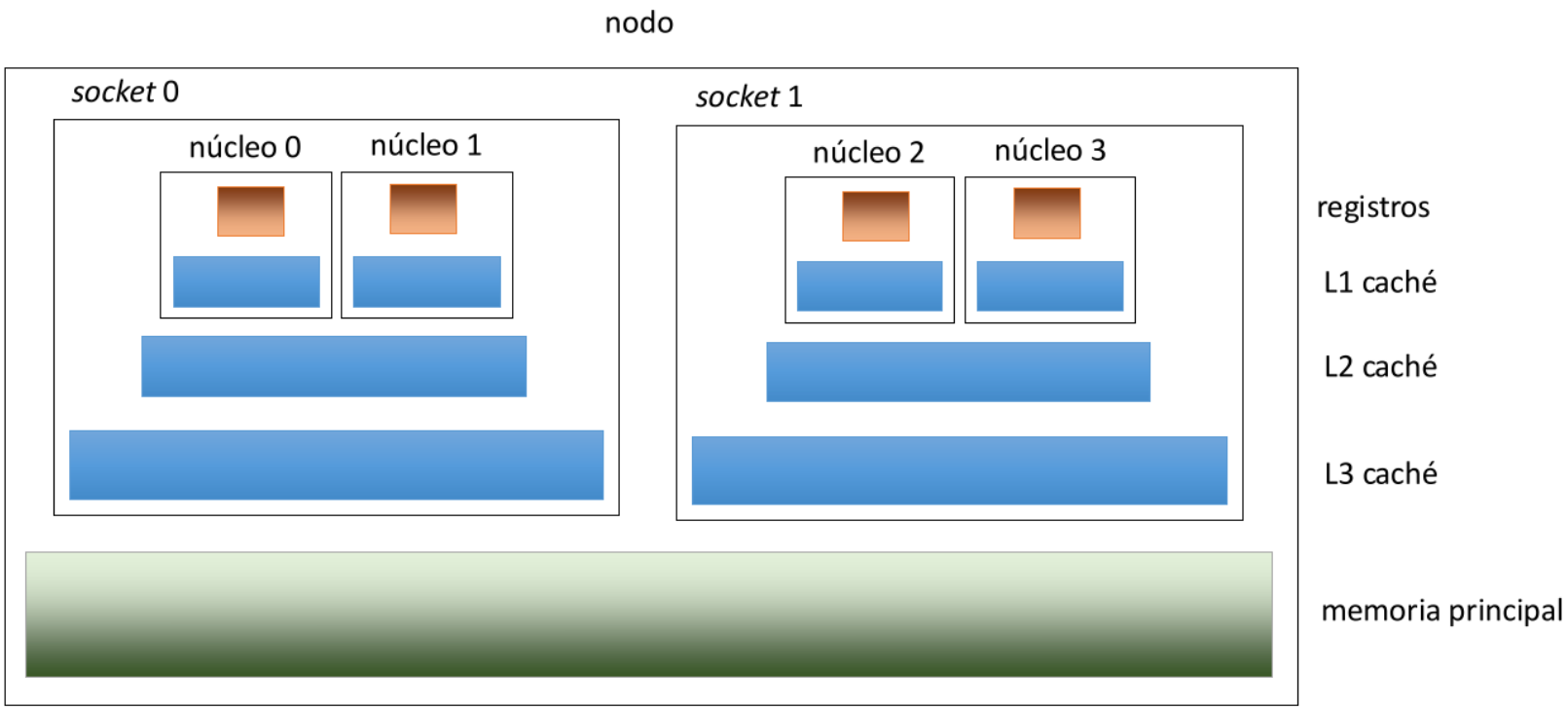
Memoria compartida:

- Todos los cores acceden a la misma RAM, dentro de un solo nodo.
- Librerías: Numba, `multiprocessing`, `joblib`.
- Limitante: la memoria RAM disponible en el nodo.

Memoria distribuida:

- Cada nodo tiene su propia memoria; se comunican por red.
- Los datos deben transferirse explícitamente entre nodos.
- Librerías: MPI, Dask, Ray.

Los procesadores *multi-core*: memoria compartida



Los procesadores *multi-core*: memoria compartida

- Los núcleos en un procesador *multi-core* pueden acceder a toda la memoria RAM.
- Hay muchas variantes:
  - Número de núcleos.
  - Caché privado o compartido.
- La arquitectura más común en un computador personal.

Vectorización en Python

- Un consejo general para programar algoritmos es hacer **vectorización**.
  - Uso eficiente de memoria y caché.
  - Normalmente resulta en *pipelining*.
  - Oportunidades de hacer *multithreading*.
- Estrategia en Python
  - Usar Numpy arrays en lugar de listas en Python.
  - Usar funcionalidad de librerías compiladas (a través de Numpy , SciPy , etc) en lugar de *for-loops* de Python.

[0] https://numpy.org/

[0] https://scipy.org/

Computación paralela

- ¿Cómo hacer *multithreading* en Python?
- Utilizando funcionalidad de librerías como Numpy.
- Programar la paralelización explícitamente.

Computación paralela

- ¿Cómo paralelizar código en lenguajes compilados como C, C++ y Fortran?
- Los compiladores son inteligentes pero no pueden paralelizar código sin directivas explícitas.
- La paralelización sigue siendo principalmente la responsabilidad del programador.
- OpenMP nos permite escribir directivas de compilador para hacer *multithreading*.

Lenguajes de programación

- Lenguajes interpretados (*scripted*)
  - Código se ejecuta línea por línea
  - Instrucciones de alto nivel
  - Python, Julia, R, etc
- Lenguajes compilados
  - Código es convertido en un programa ejecutable
  - Dos pasos: compilar y ejecutar
  - Fortran, C, C++, Rust, etc

El intérprete de Python

CPython ejecuta el código línea por línea: lo traduce a *bytecode* y lo interpreta en su máquina virtual.

- El intérprete mantiene un estado global (conteo de referencias, memoria).
- Dos hilos modificando ese estado simultáneamente pueden corromperse.
- Ante eso, CPython usa el **Global Interpreter Lock (GIL)**.

El GIL

El **Global Interpreter Lock (GIL)** permite que solo un hilo ejecute bytecode Python a la vez, incluso en sistemas multicore.

- Los *threads* de Python son útiles para I/O, pero no para cómputo.
- Código CPU-bound con múltiples *threads* puede ser más lento que secuencial.
- La alternativa es usar librerías que liberan el GIL o que compilan el código: Numpy, Numba, etc.

Librerías de computación paralela

- OpenMP.
  - En base al modelo de *multithreading*.
  - Paralelización en memoria compartida.
- MPI (*Message Passing Interface*)
  - Comunicación explícita por paso de mensajes entre procesos independientes.
  - Paralelización en memoria distribuida.
- Disponible para Fortran, C y C++.
  - Python tiene la librería MPI4Py.
  - Python no tiene una librería equivalente a OpenMP.

Procesos

Un **proceso** es un programa en ejecución, con su propio espacio de memoria.

- Al crearse, el SO le asigna su propio espacio de memoria: stack y heap.
- Cada proceso tiene su propia memoria: no comparte datos con otros procesos.
- Un proceso puede tener múltiples hilos, todos compartiendo su memoria.
- La comunicación entre procesos requiere mecanismos explícitos.

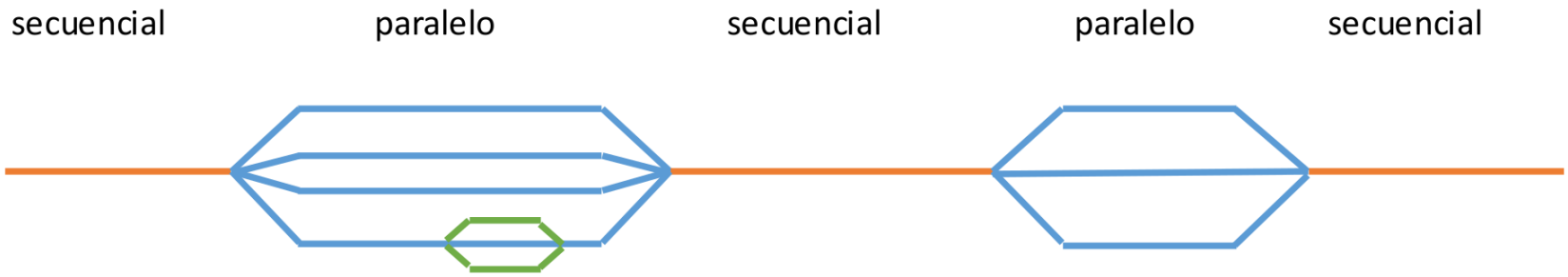
Hilos (*threads*)

Un **hilo** es la unidad mínima de ejecución dentro de un proceso.

- Varios hilos comparten el mismo espacio de memoria del proceso.
- El sistema operativo los planifica de forma independiente.
- Permiten ejecutar varias tareas *aparentemente* en paralelo dentro de un mismo programa.
- Cada *core* puede ejecutar un hilo a la vez: con N cores se pueden ejecutar N hilos simultáneamente.
- Si hay más hilos que cores, el SO los intercala.

Hilos

- Un proceso puede crear hilos dinámicamente.
- Los subprocesos tienen una memoria privada y pueden acceder a los datos en la memoria compartida.
- El mecanismo de bifurcación (*fork-join*) tiene un hilo original y crea otros hilos dinamicamente (*spawning*).



Los datos en los hilos

- Cuando se usan múltiples hilos, los datos pueden ser 'privados' o 'compartidos' (*private - shared*).
  - cuando se crea dentro de un hilo, los datos son privados.
    - los datos privados se eliminarán cuando finalice el hilo.
  - cuando se crea fuera de un hilo, los datos se comparten.
    - la memoria compartida puede ser utilizada por todos los hilos.
- Se puede indicar el estado (*private/shared*) de un dato explícitamente en OpenMP.

Numba

- La estrategia JIT (*just-in-time compilation*).
  - Compilar y ejecutar un código justo cuando se le necesita.
- La librería Numba para Python (`https://numba.pydata.org/`).
  - Código abierto.
  - Compilación JIT de código de Python.
- Instalación: `conda install numba` (importante usar miniconda!)

Numba

- El uso de JIT por parte de Numba
  - El programador escribe código de Python/Numpy.
  - Numba analiza este código.
  - Numba crea código eficiente y optimizado.
  - Numba ejecuta la función compilada.
  - Numba entrega el resultado en formato de Python/Numpy.
- Numba compila el código cuando la función es llamada por primera vez y guarda la compilación para llamadas siguientes.
  - Precaución con esto a la hora de medir el tiempo de cómputo.

Numba y LLVM

Numba usa **LLVM** (*Low Level Virtual Machine*) como motor de compilación.

- LLVM es una infraestructura de compilación de propósito general, usada también por Clang (C/C++).
- Numba traduce el código Python a *LLVM IR* (representación intermedia), que LLVM compila a código máquina nativo.
- LLVM aplica optimizaciones avanzadas: vectorización, inlining, eliminación de código muerto, etc.
- Por eso el rendimiento de Numba puede ser cercano al de C o Fortran.

Numba

- Python decorators
  - Toma una función y retorna otra función.
- Llamado a Numba.
  - Agregar `@jit` antes de una función.

Ejemplo Numba

Una función básica en Python

```
def myFunction(x):

    total = 0
    for i in range(len(x)):
        total += x[i] * x[i]

    return total
```

Ejemplo Numba

Una función básica en Python acelerada con Numba

```
1      @jit
2      def myFunction(x):
3
4         total = 0
5         for i in range(len(x)):
6            total += x[i] * x[i]
7
8         return total
```

- Si Numba logra escribir esto en código compilable, lo hace.
- Si no, usará la funcionalidad de Python.

Numba

- Numba tiene dos modos.
  - `@jit(nopython=True)` o `@njit`, no utilizar el intérprete de Python.
  - Este modo normalmente más rápido pero más restrictivo en funcionalidad, más complicado de debuggear.
- `@jit(nopython=False)` o `@jit`: si Numba no logra compilar, cae silenciosamente al intérprete de Python sin lanzar un error.
- Las últimas versiones de Numba ya no permiten el modo `nopython=False`.

Ejemplo Numba

Recomendado

Una función básica acelerada con Numba sin el intérprete de Python

```
1      @njit
2      def myFunction(x):
3
4         total = 0
5         for i in range(len(x)):
6            total += x[i] * x[i]
7
8         return total
```

- El resultado es una función compilada.

Numba

- Numba puede paralelizar código.
  - `@njit(parallel=True)`, Numba paraleliza el código bajo el paradigma de *multithreading*.
    - Numba busca oportunidades de paralelizar el código.
    - Si Numba no encuentra una manera de paralelizar, corre el código secuencialmente.
    - Numba intenta solucionar las condiciones de carrera pero no hay garantía de consistencia.
- Paralelizar ciclos a mano con `prange()`.

Ejemplo Numba Paralelizado

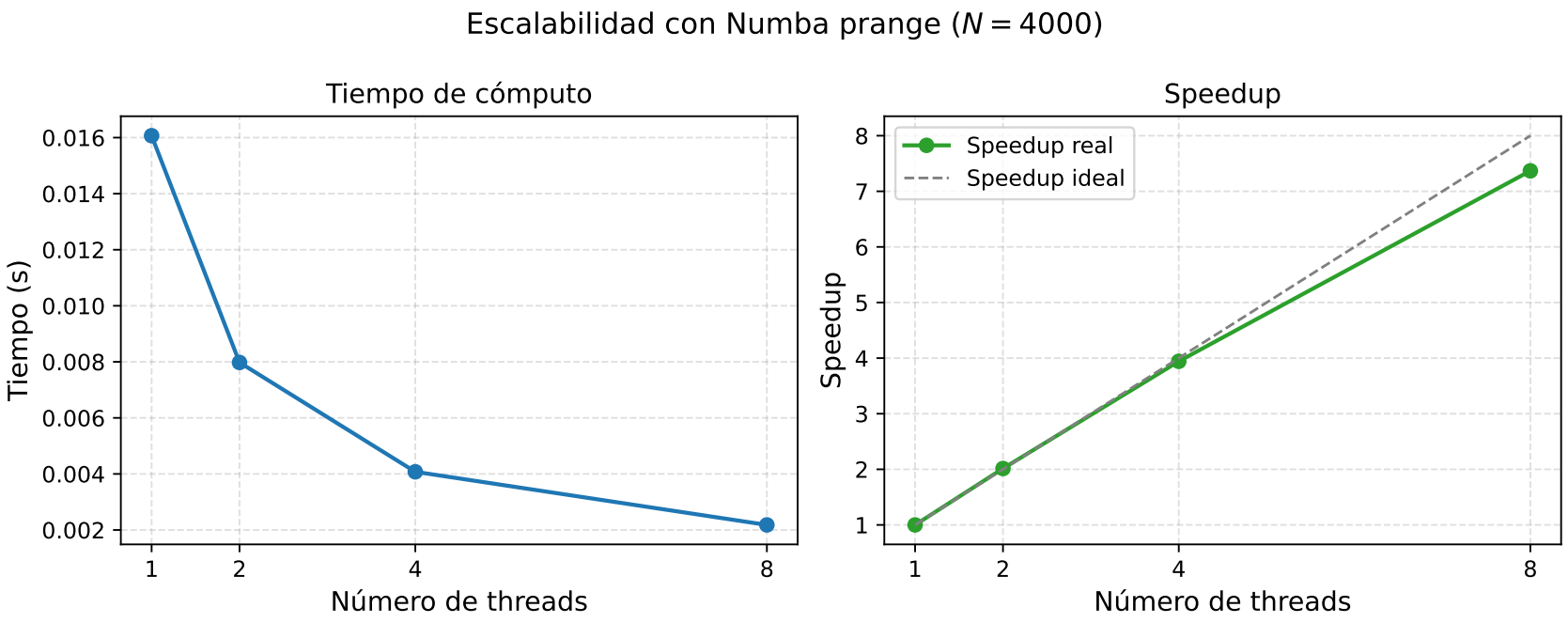
Recomendado

Una función básica acelerada con Numba sin el intérprete de Python

```
1   from numba import njit, prange
2
3   @njit(parallel=True)
4   def myFunction(x):
5
6       total = 0
7       for i in prange(len(x)):
8           total += x[i] * x[i]
9
10      return total
```

- El resultado es una función compilada y paralelizada.
- Es posible manejar el número de *threads* con `numba.set_num_threads(N)`.
- Es capaz de manejar la condición de carrera.

Escalabilidad con prange



Ejemplo: suma del producto exterior

Calcular $\sum_{i,j} x_i x_j$, usando distintas implementaciones:

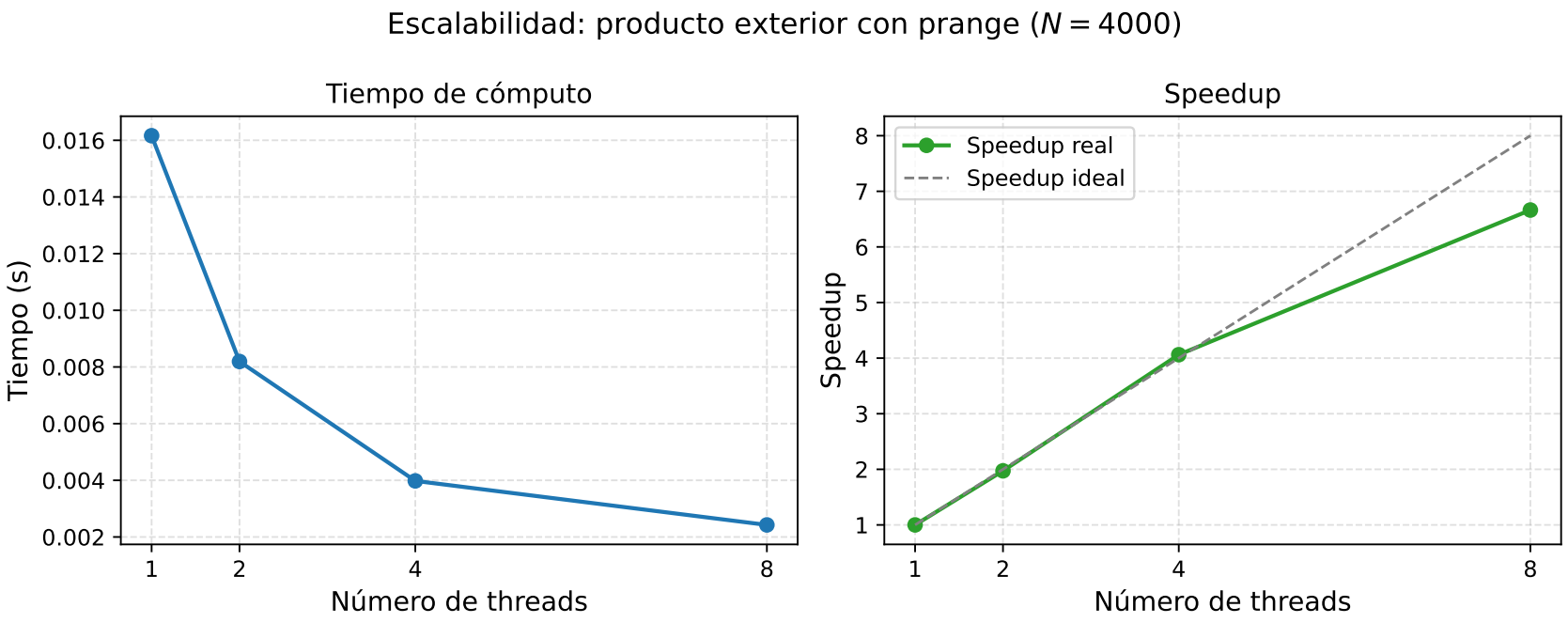
```
def slow_function(x):        # Python puro, O(N^2)
    total = 0
    for i in range(len(x)):
        for j in range(len(x)):
            total += x[i] * x[j]
    return total

@njit
def fast_function(x):        # Numba compilado, O(N^2)
    total = 0
    for i in range(len(x)):
        for j in range(len(x)):
            total += x[i] * x[j]
    return total
```

Ejemplo: suma del producto exterior

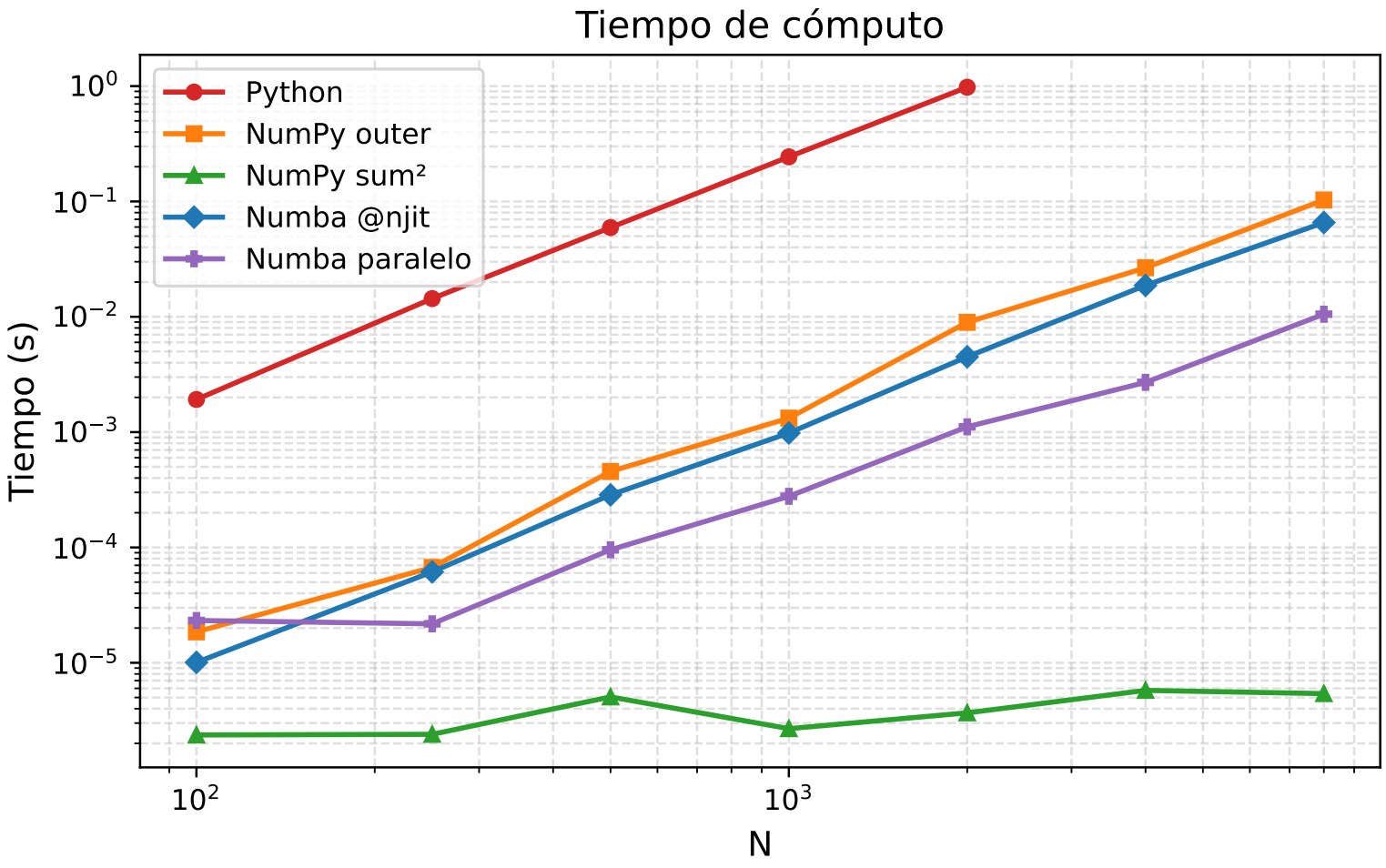
```python
@njit(parallel=True)
def parallel_function(x):   # Numba paralelo, O(N^2)
    total = 0
    for i in prange(len(x)):
        for j in prange(len(x)):
            total += x[i] * x[j]
    return total

def numpy_fast(x):          # NumPy, O(N)
    return np.sum(x) ** 2
```

Nota: $\sum_{i,j} x_i x_j = \left(\sum_i x_i\right)^2$, reduciendo el problema de $O(N^2)$ a $O(N)$.

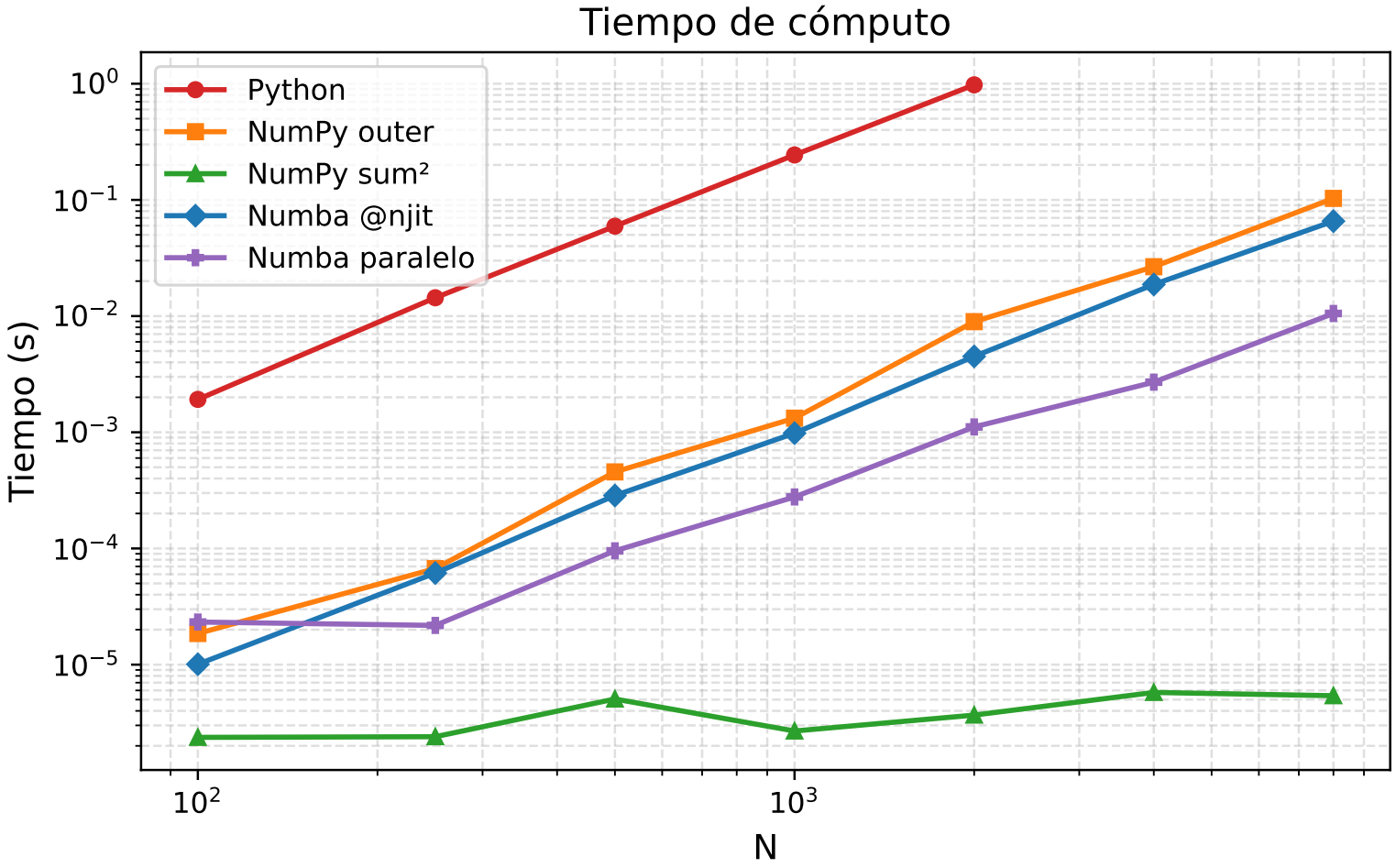
Escalabilidad: producto exterior con `prange`



Ejemplo: suma del producto exterior



Ejemplo: suma del producto exterior



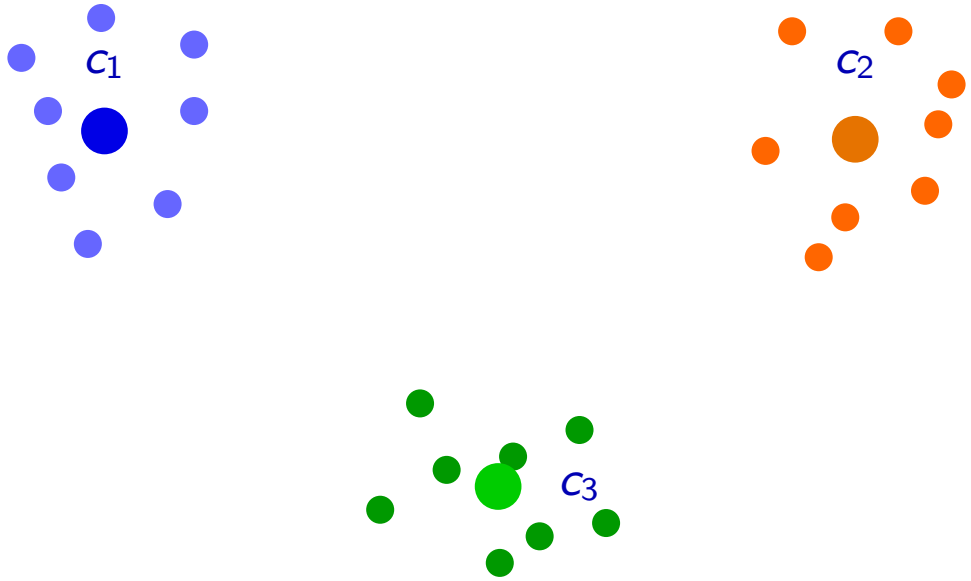
$$\sum_{i,j} x_i x_j = \left(\sum_i x_i\right)^2$$: el **algoritmo** importa más que la implementación.

Algoritmo K-means

K-means agrupa N puntos en K clusters buscando los **centroides** que minimizan la distancia intra-cluster.

Cada iteración tiene dos etapas:

- **Asignación:** cada punto se asigna al centroide más cercano.
- **Actualización:** cada centroide se recalcula como el promedio de los puntos asignados.

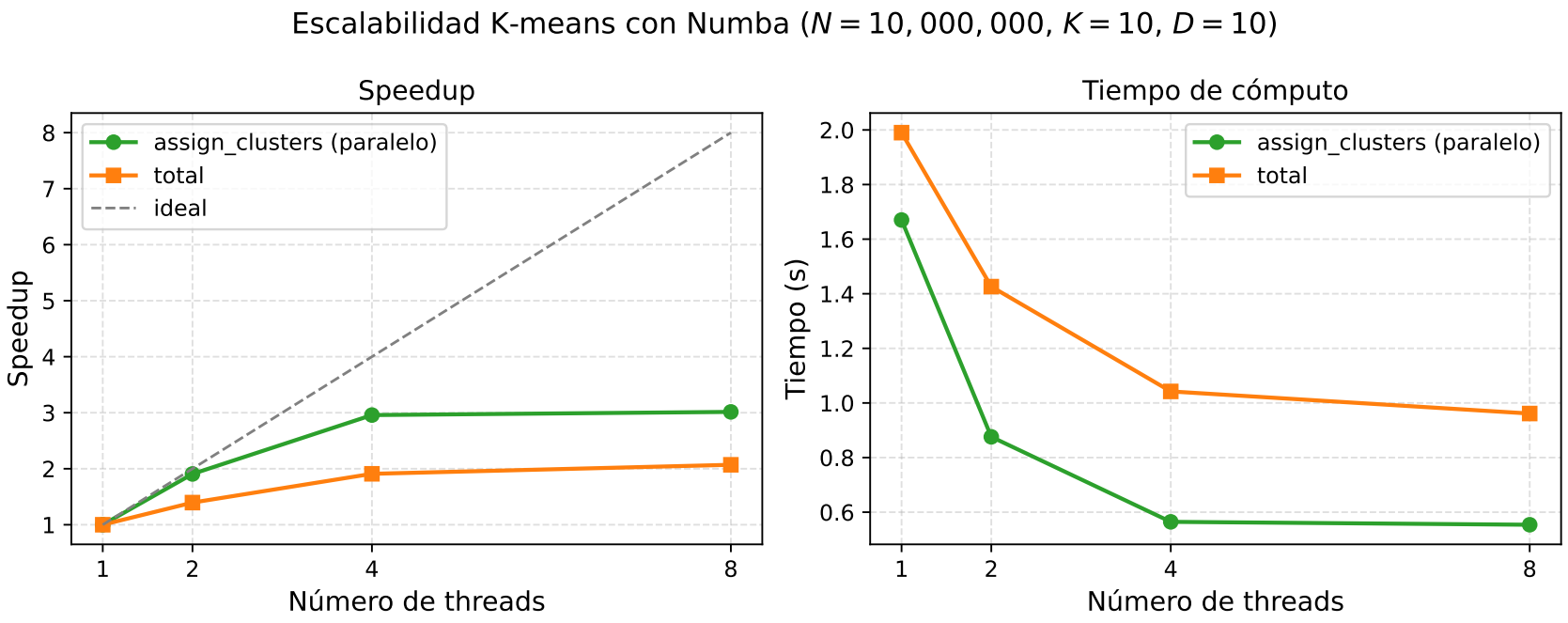


Ejemplo: K-means con Numba

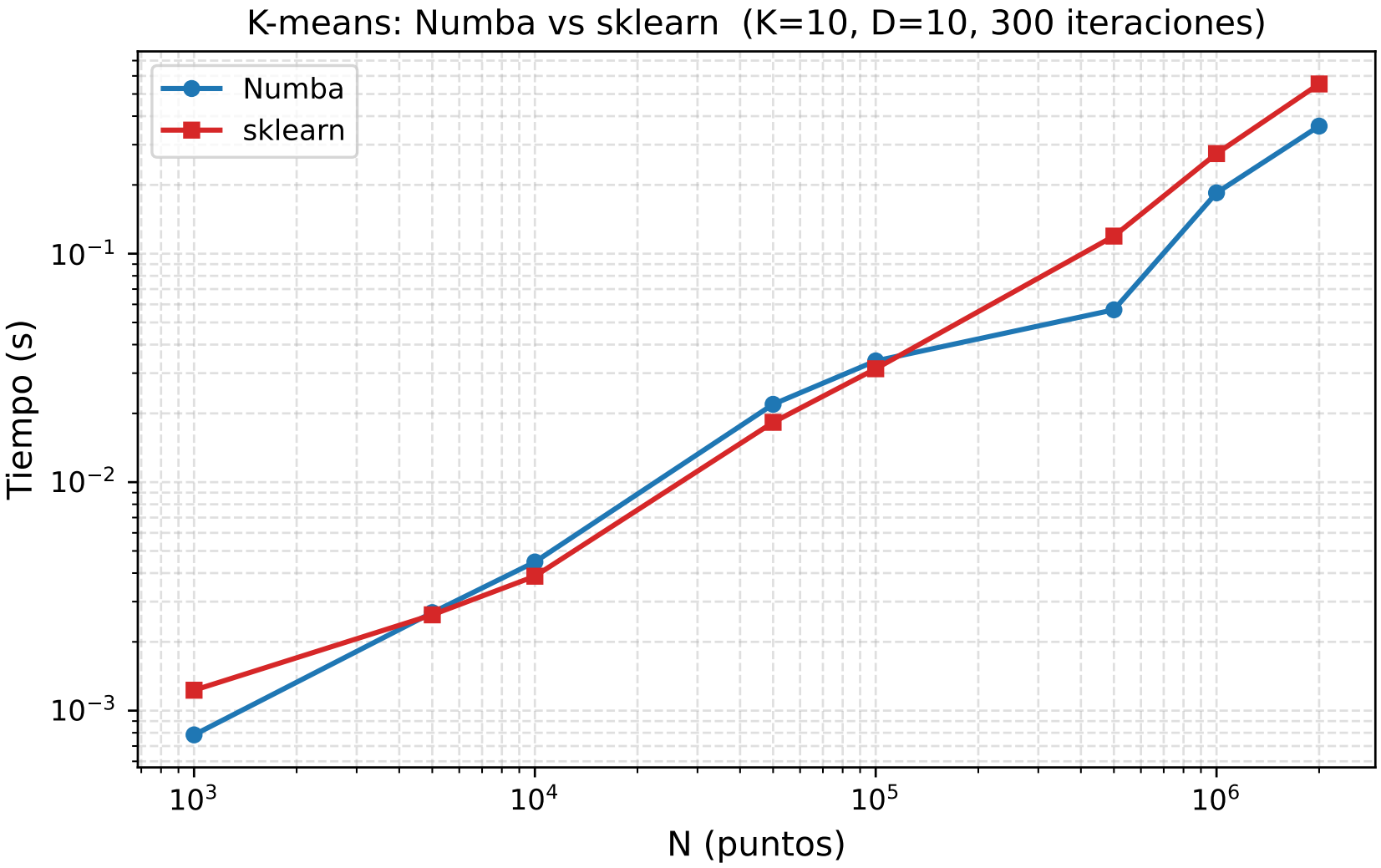
El paso más costoso es la asignación de puntos a clusters:

```
@njit(parallel=True)
def assign_clusters(X, centroids):
    N, D = X.shape
    K = centroids.shape[0]
    labels = np.empty(N, dtype=np.int32)
    for i in prange(N):         # paralelo sobre puntos
        min_dist = np.inf
        for k in range(K):
            dist = 0.0
            for d in range(D):
                diff = X[i, d] - centroids[k, d]
                dist += diff * diff
            if dist < min_dist:
                min_dist = dist
                labels[i] = k
    return labels
```

Ejemplo: K-means con Numba

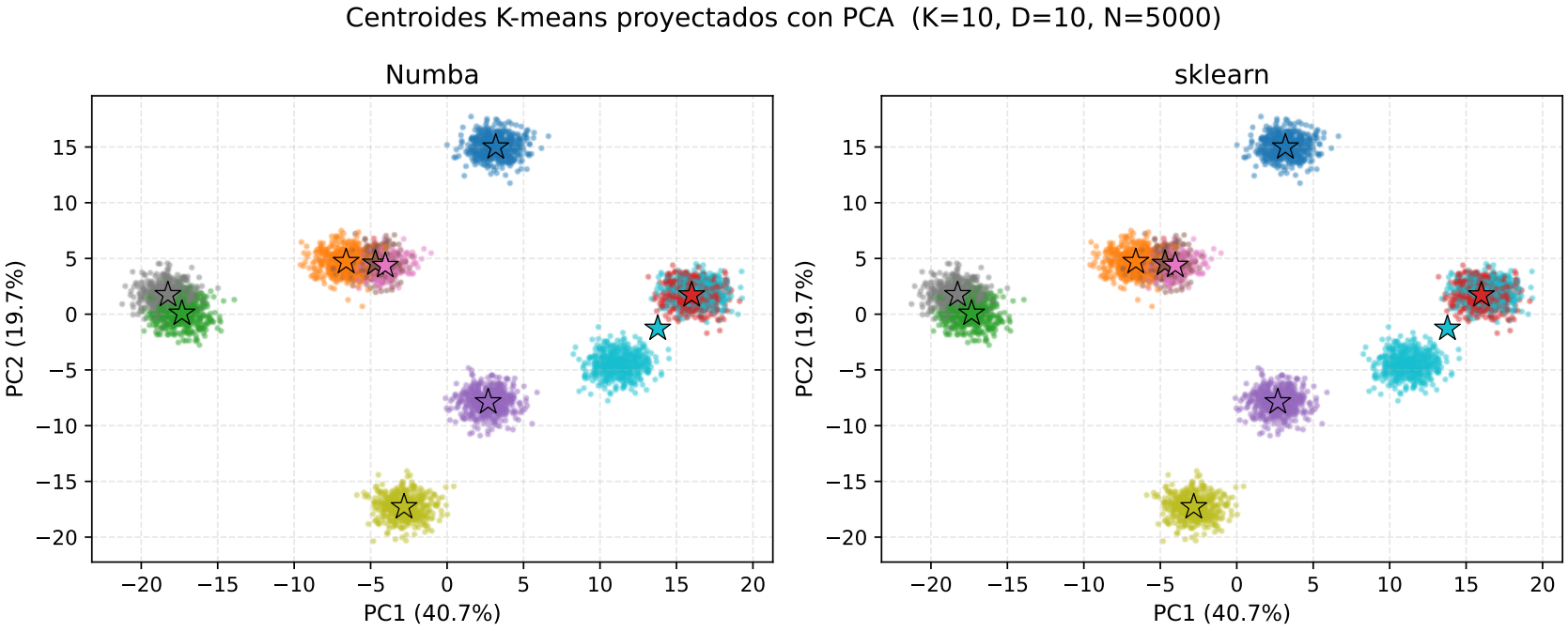


Ejemplo: K-means con Numba



Ejemplo: K-means con Numba

Centroides proyectados con PCA: ambos métodos convergen a la misma solución.



Numba

- Numba tiene sus límites.
  - A veces desafiante de debuggear.
  - Solo puede interpretar una submuestra de la funcionalidad de Python.
    - maneja principalmente valores (*int*, *float*, *array*).
    - No maneja objetos como *dict*.
  - No es compatible con todas las librerías externas: maneja gran parte de Numpy y Scipy, pero no todas.
- Numba es eficiente en acelerar *for-loops* y la estructura *if-else*.

Numba

- Numba también se puede usar como decorador de clases: `@jitclass()`.
- Numba tiene funcionalidad para programar en tarjetas gráficas: `@cuda.jit`
  - Requiere instalación de CUDA.

Clase 11: *Multithreading* en Python – Numba

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Jueves 16 de Abril del 2026